

Appendix A Search string

Table 1: Search database and search string

Search database	Search string
Scopus	(TITLE-ABS-KEY ("software reus*")) AND TITLE-ABS-KEY ("literature review" OR "mapping study") AND (LIMIT-TO (SUBJAREA, "COMP"))
IEEE	("All Metadata": "software reus*" AND ("All Metadata": "literature review" OR "mapping study"))
WOS	((AK=(software reus*)) AND AK=(literature review OR mapping study) AND (AB=(software reus*)) AND AB=(literature review OR mapping study) AND (TI=(software reus*)) AND TI=(literature review OR mapping study))
ACM	[Title: software reus*] AND [[Title: literature review] OR [Title: mapping study]] AND [Abstract: software reus*] AND [[Abstract: literature review] OR [Abstract: mapping study]] AND [Keywords: software reus*] AND [[Keywords: literature review] OR [Keywords: mapping study]]

Appendix B Quality assessment checklist

Table 2: Quality assessment criteria

Quality assessment criteria	Scale
Criteria 1. Is this a research paper? Consider: –Is the paper based on research (or is it merely a “lessons learned” report based on expert opinion)?	Yes ☐ No ☐
Criteria 2. Is there a clear statement of the aims of the research? Consider: –Is there a rationale for why the study was undertaken? –Is the study’s focus or main focus on software reuse costs and/or benefits? –Does the study present empirical data? –Is there a clear statement of the study’s primary outcome (i.e., costs, benefits, metrics)?	Yes ☐ No ☐
Criteria 3. Is there an adequate description of the context in which the research was carried out? Consider whether the researcher has identified: –The context information about the product used (e.g., maturity, quality, size, system type, customization, programming language). –The context information about the process (e.g., activities, workflow, artifacts). –The context information about practices, tools, and techniques (e.g., CASE tools, practices and techniques). –The information of the people and team involved in the study (e.g., roles, experience). –The context information about the company/organization (e.g., model of the overall organization, organization unit, certificate, distribution, domain of the organization)	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 4. Research design Was the research design appropriate to address the aims of the research? Consider: – Have they discussed and justified how they decided which type of research method to use? Write down the research method that the authors reported. – Is the method appropriate or not? If you agree with the author regarding the written research method, write yes. If no, write down the method that you think they used.	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 5. Sampling Was the recruitment strategy appropriate to the aims of the research? Consider: –Has the researcher explained how the participants or cases were identified and selected? –Are the cases defined and described precisely? –Were the cases representative of a defined population? –Have the researchers explained why the participants or cases they selected were the most appropriate to provide access to the type of knowledge sought by the study? –Was the sample size sufficiently large?	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐

Table 2 continued from previous page

Quality assessment criteria	Scale
Criteria 6. Control group Was there a control group with which to compare treatments? Consider: –How were the controls selected? –Were they representative of a defined population? –Was there anything special about the controls? –Was the non-response high? Could non-respondents be different in any way?	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 7. Data collection Was the data collected in a way that addressed the research issue? Consider: –Were all measures clearly defined (e.g., unit and counting rules)? –Is it clear how data was collected (e.g., semi-structured interviews, focus groups, etc.)? –Has the researcher justified the methods that were chosen? –Has the researcher made the methods explicit (e.g., is there an indication of how interviews were conducted, did they use an interview guide)? –If the methods were modified during the study, has the researcher explained how and why? –Whether the form of the data is clear (e.g., tape recording, video material, notes, etc.) –Whether quality control methods were used to ensure completeness and accuracy of data collection	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 8. Data analysis Was the data analysis sufficiently rigorous? Consider: –Was there an in-depth description of the analysis process? –If thematic analysis was used, is it clear how the categories or themes were derived from the data? –Has sufficient data been presented to support the findings? –To what extent has contradictory data been taken into account? –Whether quality control methods were used to verify the results	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 9. Reflexivity (research partnership relations/recognition of researcher bias) Has the relationship between researcher and participants been considered adequately? Consider: –Did the researcher critically examine their own role, potential bias and influence during the formulation of research questions, sample recruitment, data collection, and analysis and selection of data for presentation? –How the researcher responded to events during the study and whether they considered the implications of any changes in the research design.	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐

Table 2 continued from previous page

Quality assessment criteria	Scale
Criteria 10. Findings Is there a clear statement of findings? Consider: –Are the findings explicit (e.g., the magnitude of effect)? –Has an adequate discussion of the evidence, both for and against the researcher’s arguments, been demonstrated? –Has the researcher discussed the credibility of their findings (e.g., triangulation, respondent validation, more than one analyst)? –Are the limitations of the study discussed explicitly? –Are the findings discussed in relation to the original research questions? –Are the conclusions justified by the results?	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐
Criteria 11. Value of the research Is the study of value for research or practice? Consider: –Does the researcher discuss the contribution the study makes to existing knowledge or understanding (e.g., do they consider the findings in relation to current practice or relevant research-based literature)? –Does the research identify new areas in which research is necessary? –Does the researcher discuss whether or how the findings can be transferred to other populations, or consider other ways in which the research can be used?	Yes ☐ Partial - 0.25 ☐ Partial - 0.5 ☐ Partial -0.75 ☐ No ☐

Appendix C Data extraction table

Table 3: Data extraction form

Extracted data type	No.	Data item name	Description	Related RQ
General information	D1	Publication venue	To know which venue is the paper published.	RQ1
	D2	Year	To know when the paper is published.	RQ1
Information about the sample, population or participants – software reuse context	D3	Reuse approach	To know which reuse approach is used in the industry, e.g., domain engineering.	RQ1
	D4	Reuse type - modification	To know how the reusable assets in code are being reused in the industry, verbatim or modified (black-box reuse or white-box reuse).	RQ1
	D5	Reuse purpose	To know the type of the reusable assets, infrastructure reuse or domain reuse.	RQ1
	D6	Reuse method	To know if it is systematic reuse or not. If it is systematic reuse, what are the related activities or workflows are used in the industry.	RQ1
	D7	Development methodology	To know how the organizations divide the software development work, particularly when conducting software reuse, e.g., Agile development, Continuous integration, Incremental development, Rapid application development, Waterfall development.	RQ1
	D8	Size of the organizations	To know the size of the organizations, e.g., number of employees.	RQ1
	D9	Size of the applications	To know the size of the applications, e.g. based on number of lines of code.	RQ1

Table 3 continued from previous page

Extracted data type	No.	Data item name	Description	Related RQ
	D10	Reuse comparison scheme	To know how reuse impacts are compared.	RQ1
Central focus of the study	D11	Benefits and its metrics	To know the software reuse benefits that observed in the industry. To know the measurements of software reuse benefits that applied or have the potential to be applied in the industry.	RQ2, RQ3
	D12	Costs and its metrics	To know the software reuse costs that faced in the industry. To know the measurements of software reuse costs that applied or has potential to be applied in the industry.	RQ2, RQ3
Information about the sample, population or participants – empirical background	D13	Study type	To know the study type of the publications, i.e. quantitative study, qualitative study or experience report.	RQ4
	D14	Threats to validity	1. To know if the threats to validity are reported in the paper. 2. To know if the threats to validity are appropriate. 3. To know if there are uncovered threats to validity.	RQ4

Appendix D List of primary studies

Table 4: Included primary studies

ID	Primary studies
QN ⁶ 1 [1]	Succi, Giancarlo, Luigi Benedicenti, and Tullio Vernazza. "Analysis of the effects of software reuse on customer satisfaction in an RPG environment." <i>IEEE Transactions on Software Engineering</i> 27, no. 5 (2001): 473-479.
ER2 [7]	Lim, Wayne C. "Effects of reuse on quality, productivity, and economics." <i>IEEE software</i> 11, no. 5 (1994): 23-30.
QN3 [12]	Thomas, William M., Alex Delis, and Victor R. Basili. "An analysis of errors in a reuse-oriented development environment." <i>Journal of Systems and Software</i> 38, no. 3 (1997): 211-224.
QN4 [2]	Morisio, Maurizio, Daniele Romano, and Ioannis Stamelos. "Quality, productivity, and learning in framework-based development: an exploratory case study." <i>IEEE Transactions on Software Engineering</i> 28, no. 9 (2002): 876-888.
QN5 [29]	Selby, Richard W. "Enabling reuse-based software development of large-scale systems." <i>IEEE Transactions on Software Engineering</i> 31, no. 6 (2005): 495-510.
ER6 [14]	Morad, Shlomit, and Tsvi Kuflik. "Conventional and open source software reuse at Orbotech-an industrial experience." In <i>IEEE International Conference on Software-Science, Technology & Engineering (SwSTE'05)</i> , pp. 110-117. IEEE, 2005.
QN7 [46]	Frakes, William B., and Giancarlo Succi. "An industrial study of reuse, quality, and productivity." <i>Journal of Systems and Software</i> 57, no. 2 (2001): 99-106.
ER8 [10]	Tomer, Amir, Leah Goldin, Tsvi Kuflik, Esther Kimchi, and Stephen R. Schach. "Evaluating software reuse alternatives: a model and its application to an industrial case study." <i>IEEE Transactions on Software Engineering</i> 30, no. 9 (2004): 601-612.
QN9 [13]	Mohagheghi, Parastoo, Reidar Conradi, Ole M. Killi, and Henrik Schwarz. "An empirical study of software reuse vs. defect-density and stability." In <i>Proceedings. 26th International Conference on Software Engineering</i> , pp. 282-291. IEEE, 2004.
QN10 [4]	Baldassarre, Maria Teresa, Alessandro Bianchi, Danilo Caivano, and Giuseppe Visaggio. "An industrial case study on reuse oriented development." In <i>21st IEEE International Conference on Software Maintenance (ICSM'05)</i> , pp. 283-292. IEEE, 2005.
QN11 [3]	Zhang, Weishan, and Stan Jarzabek. "Reuse without compromising performance: industrial experience from RPG software product line for mobile devices." In <i>Software Product Lines: 9th International Conference, SPLC 2005, Rennes, France, September 26-29, 2005. Proceedings</i> 9, pp. 57-69. Springer Berlin Heidelberg, 2005.
QN12 [15]	Ha, Waileung, Hongyi Sun, and Min Xie. "Reuse of embedded software in small and medium enterprises." In <i>2012 IEEE International Conference on Management of Innovation & Technology (ICMIT)</i> , pp. 394-399. IEEE, 2012.
QN13 [21]	Mohagheghi, Parastoo, and Reidar Conradi. "An empirical investigation of software reuse benefits in a large telecom product." <i>ACM Transactions on Software Engineering and Methodology (TOSEM)</i> 17, no. 3 (2008): 1-31.

⁶QN: quantitative studies. ER: experience reports. QL: qualitative studies. QNL: both qualitative and quantitative studies.

Table 4 continued from previous page

ID	Primary studies
QL14 [16]	Slyngstad, Odd Petter N., Anita Gupta, Reidar Conradi, Parastoo Mohagheghi, Harald Rønneberg, and Einar Landre. "An empirical study of developers views on software reuse in statoil asa." In Proceedings of the 2006 ACM/IEEE international symposium on empirical software engineering, pp. 242-251. 2006.
QN15 [27]	Deniz, Berkhan, and Semih Bilgen. "An empirical study of software reuse and quality in an industrial setting." In Computational Science and Its Applications-ICCSA 2014: 14th International Conference, Guimarães, Portugal, June 30-July 3, 2014, Proceedings, Part V 14, pp. 508-523. Springer International Publishing, 2014.
ER16 [5]	Quilty, Gerard, and Mel Ó. Cinnéide. "Experiences with software product line development in risk management software." In 2011 15th International Software Product Line Conference, pp. 251-260. IEEE, 2011.
ER17 [9]	Beyer, Hans-Jörg, Dirk Hein, Clemens Schitter, Jens Knodel, Dirk Muthig, and Matthias Naab. "Introducing architecture-centric reuse into a small development organization." In High Confidence Software Reuse in Large Systems: 10th International Conference on Software Reuse, ICSR 2008, Beijing, China, May 25-29, 2008 Proceedings 10, pp. 1-13. Springer Berlin Heidelberg, 2008.
ER18 [53]	Sellier, David, Mike Mannion, G. Benguria, and G. Urchegui. "Introducing software product line engineering for metal processing lines in a small to medium enterprise." In 11th International Software Product Line Conference (SPLC 2007), pp. 54-62. IEEE, 2007.
ER19 [8]	Henry, Emmanuel, and Benoit Faller. "Large-scale industrial reuse to reduce cost and cycle time." IEEE Software 12, no. 5 (1995): 47-53.
QNL20 [57]	Goldin, Leah, and Daniel M. Berry. "Reuse of requirements reduced time to market at one industrial shop: a case study." Requirements Engineering 20 (2015): 23-44.
QN21 [47]	Gupta, Anita, Jingyue Li, Reidar Conradi, Harald Rønneberg, and Einar Landre. "A case study comparing defect profiles of a reused framework and of applications reusing it." Empirical Software Engineering 14 (2009): 227-255.
ER22 [54]	Kodama, Ryuichiro, Jun Shimabukuro, Yoshimitsu Takagi, Shinobu Koizumi, and Shun'ichi Tano. "Experiences with commonality control procedures to develop clinical instrument system." In Proceedings of the 18th International Software Product Line Conference-Volume 1, pp. 254-263. 2014.
QN23 [48]	Kolb, Ronny, Isabel John, Jens Knodel, Dirk Muthig, Uwe Haury, and Gerald Meier. "Experiences with product line development of embedded systems at testo ag." In 10th International Software Product Line Conference (SPLC'06), pp. 10-pp. IEEE, 2006.
ER24 [6]	Otsuka, Jun, Kouichi Kawarabata, Takashi Iwasaki, Makoto Uchiba, Tsuneo Nakanishi, and Kenji Hisazumi. "Small inexpensive core asset construction for large gainful product line development: developing a communication system firmware product line." In Proceedings of the 15th International Software Product Line Conference, Volume 2, pp. 1-5. 2011.
QN25 [49]	Sun, Hongyi, Waileung Ha, Min Xie, and Jianglin Huang. "Modularity's impact on the quality and productivity of embedded software development: a case study in a Hong Kong company." Total Quality Management & Business Excellence 26, no. 11-12 (2015): 1188-1201.
QN26 [50]	Mao, Fagui, Xuyang Cai, Beijun Shen, Yong Xia, and Bo Jin. "Operational pattern based code generation for management information system: An industrial case study." In 2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), pp. 425-430. IEEE, 2016.

Table 4 continued from previous page

ID	Primary studies
ER27 [55]	Powell, Antony, and Duncan Brown. "A practical strategy for industrial reuse improvement." <i>Software Process: Improvement and Practice</i> 4, no. 3 (1998): 173-182.
QN28 [51]	Rothenberger, Marcus A., and Kevin J. Dooley. "A performance measure for software reuse projects." <i>Decision Sciences</i> 30, no. 4 (1999): 1131-1153.
QN29 [52]	Mihale-Wilson, Cristina, Patrick Felka, Oliver Hinz, and Martin Spann. "The Impact of Strategic Core-Component Reuse on Product Life Cycles: The Case of Location-Based Games." <i>Business & Information Systems Engineering</i> 64, no. 2 (2022): 223-237.
QL30 [56]	Agresti, William W. "Software reuse: developers' experiences and perceptions." <i>Journal of Software Engineering and Applications</i> 4, no. 01 (2011): 48.

Appendix E Relation between software reuse impacts and other characteristics

The projects the primary studies investigated vary in size (see Figure 1). We found better quality, improved productivity, and cost-saving are observed in all types of project sizes - the majority of them (77%) are medium- and large-scale projects. Some less frequently investigated benefits - improved learning, increased performance, prolonged lifecycle of reused product (game), and standardized architecture, were only reported in the primary studies investigating small and medium projects. The results indicate no clear evidence of which project size may cause more software reuse costs.

Figure 1 also shows the relation between software reuse costs, benefits, reuse purpose, and measurement level. For reuse purposes, the results show that domain reuse covers the most identified software reuse benefits, indicating it may more likely benefit companies. However, we cannot say infrastructure reuse and architecture reuse will less likely benefit companies since they are way less investigated. The same goes for the relation between software reuse costs and reuse purposes. In addition, we did not find a single primary study that compares the software reuse impact between different reuse purposes. As for the measurement level, the results showed that all process and team related benefits are measured at the project level, except improved productivity. Product-related benefits are measured similarly in both measurement levels. We found that except for learning and architecture costs, the remaining three identified software reuse costs are measured both at the project and module levels.



Figure 1: Software reuse costs, benefits, project sizes, reuse purposes, and measurement levels

Appendix F Metrics

Table 5: Metrics for better quality and their definition and results

Metrics	Definition	Results
Defect/Fault /Error density	ER2 (1994): Defects per thousand non-comment source statements in reused code; defects per thousand non-comment source statements in new code; defects per thousand non-comment source statements in reused and new code. QN3 (1997): Errors per thousand source statements. QN5 (2005): The number of faults divided by the number of source lines of code. QN7 (2001): Number of errors per number of non-commentary source lines (NCSL). NCSL for C and C++ is counted in the source file as the number of semicolons that are not in the comments. QN9 (2004): The number of trouble reports (TRs) per KLOC (kilo lines of non-commented code); the number of TRs per MKLOC (modified KLOC). QN12 (2012): The number of defects or errors per non-commentary source statements. The defects here refer to all pre-release defects. QN13 (2008): Number of trouble reports (TRs) per thousand lines of code. Or number of TRs per modified thousand lines of code. Modified thousand lines of code mean the size of the modified code between releases. QN21 (2009): Divided each system's Non-commented Source Lines of Code (NSLOC) by the number of defects to calculate the defect density. The NSLOC was counted using the Eclipse tool because that is the development tool used in the company. QN25 (2015): The number of defects per kilo of non-comment source statements (KNCSS). The case company has two gates for detecting defects, one from the internal teams and the other from the external customers. Here, the number of defects includes both detected from internal and external gates. QN26 (2016): The number of defects per KLOC.	ER2: One case division got 51% defect reduction, and the other case division got 24% defect reduction after reuse from previous projects. QN3: The authors found that verbatim reuse has significantly lower error density (more than a 90% reduction) compared to new development, and slight modification reuse also has a 50% reduction in error density compared to new development. Slight modification reuse also improves error density in late development - a 70% reduction. QN5: The authors investigated the relationship between the fault rate and module origin. The module has four origins: new development, major revision, slight revision, and complete reuse. The results showed that the modules reused without revision had the fewest faults and fewest faults per source line (simultaneous < 0.05). QN7: The authors investigated 12 modules. Modules varied in size from 50 to 3197 NCSL, with a median value of 679. The number of errors varied from 0 to 3 with a median value of .5. There are high negative Spearman's rank correlations between (External Reuse Frequency) ERF and Error Density (-0.62), low negative Spearman's rank correlations between (External Reuse Level) ERL and Error Density (-0.04), indicating that more reuse is related to correct software. QN9: Reused components have lower defect density than non-reused ones (about 50% less). QN12: The authors investigated 30 projects, with verbatim reuse rates ranging from 0 to 96%, and defect density ranging from 0.11 to 7.5. The r value of Pearson's coefficient between defect density and verbatim reuse rate is -0.433, indicating defect density will reduce when the reuse rate increases. The p-value of Pearson's coefficient between defect density and verbatim reuse rate is 0.014, showing a high confidence level of the result. [1] QN13: The authors investigate three releases of one product. There were 29 reused (mean size of 10.19 KLOC) and 20 non-reused blocks (mean size of 9.13 KLOC) in Release 3. Compared to the fault densities of these components and using mean values, the fault density for reused components is 44% to 61% for the non-reused ones (less difference for modified code). QN21: The authors investigated one reused framework - Java Enterprise Framework (JEF) and two applications - Digital Cargo Files (DCF) and Shipment and Allocation (S&A), that was reusing the framework. The results showed the reused software (JEF) has a lower defect density than one application (DCF) that is reusing it and has a slightly higher defect density than the other (S&A), which indicates that the systematic reuse has helped the company reduce the defect densities of the reused software. The defect density of JEF: is 11.1 per kilo NSLOC. The defect density of DCF: is 17 per kilo NSLOC. The defect density of S&A is 10.1 per kilo NSLOC. QN25: The authors investigated 30 projects, with reuse rates (software modularity rate) ranging from 0 to 1 and defect density ranging from 0.11 to 7.5. The r-value of Pearson's coefficient between defect density and verbatim reuse rate is -0.461, and the p-value of Pearson's coefficient between defect density and verbatim reuse rate is 0.006, indicating defect density will reduce when the reuse rate increases. QN26: The defect density and the average density decrease by 2.2 after adopting the reuse approach. QN15: The authors investigated 12 common platform components in three products - Components 1-8 are common in all three products; Component 11 is partly common; and Components 9, 10, and 12 are new components (i.e., they are used in corresponding products for the first time). For Components 9, 10, and 12, defect counts are more than 20. Components 9, 10, and 12 are used for the first time, and the authors observed a similar distribution as the defect counts of the common components in the first product they used. Component 11 is partially common in products 1 and 3. In Product 1, more than 50 defects are detected, and in Product 3, almost ten defects are detected. The defect distribution of this component is similar to the common components' distribution of Products 1 and 2. As components are reused in several products, the authors observed that the defect counts decreased significantly, as did the fault-proneness. ER16: The authors found the project developed with the configurable approach (35%) had fewer bugs than the customizable approach (65%).
Number of defects/bugs	QN15 (2014): The number of defects. ER16 (2011): The number of bugs in configurable and customizable approaches.	

Table 5 continued from previous page

Metrics	Definition	Results
	ER16 (2011): The number of bugs of different bug priorities - critical, serious, and trivial, in configurable and customizable approaches.	ER16: The authors found the project developed with a configurable approach had fewer bugs in all bug priorities than the one developed with a customizable approach. Critical bugs (configurable and customizable approach): about 25% and 75%. Serious bugs (reuse approach and non-reuse approach): about 45% and 55%. Trivial bugs (configurable and customizable approach): about 20% and 80%.
	ER24 (2011): The number of defects.	ER24: The number of defects found in the successive product was just one-tenth of those discovered in the first product. According to past experience, if product line development had not been performed, the ratio could be as much as one-third.
Complexity	QN10 (2005): Mean Cyclomatic Complexity of the system.	QN10: A lower trend in mean cyclomatic complexity (MCC) has been pointed out within the entire observation period in the projects developed with reuse. MCC for development with reuse: around 4. MCC for development without reuse: around 6.
	QN15 (2014): McCabe Cyclomatic Complexity (McCabeCC): the number of flows in a part of the code. Percent Branch Statements (% Branches): percentage of the statements that create a break in the sequential execution of statements. Weighted Methods for Class (WMC): number of methods defined in class. Nested Block Depth (NBD): the extent of nested blocks of code.	QN15: The authors investigated three modules with different reuse rates: 0%, 81%, and 52%. McCabeCC for the corresponding three modules is 2.49, 1.30, and 1.59. % Branches for the corresponding three modules are 18.2, 7.4, and 8.9. WMC for the corresponding three modules is 4.777778, 2.166667, and 2.9. NBD for the corresponding three modules is 1.71, 0.84, and 1.10. The use of a reuse engine causes a reduction in complexity.
	ER17 (2008): McCabe maximum nesting of control structures: maximum nesting of control structures (e.g., switch, do, while, if, and for statements). McCabe Cyclomatic Complexity (McCabeCC): the number of flows in a part of the code.	ER17: Systematic reuse has 27% decreased in maximum nesting of control structures. Systematic reuse has a 38% decrease in cyclomatic complexity. The results showed that systematic reuse helps to improve code complexity.
Coupling	QN11 (2005): Coupling Between Objects (CBO).	QN11: The authors found coupling between objects in the new product line derived games has been improved: CBO for gaming packages after using the reuse approach has reduced 1.
	QN15 (2014): Coupling Between Objects (CBO).	QN15: The authors investigated three modules with different reuse rates: 0%, 81%, and 52%. CBO for the corresponding three modules is 2.311111, 1.944444, and 2.166667. The authors observed coupling improves when the reuse rate increases.
Customer complaint density	QN1 (2001): The ratio of the number of customer complaints regarding that code file and the file size (lines of code).	QN1: The authors found customer complaint density decreases when reusing more from the domain library. Customer complaint density in the project without reuse: 258/99217. Customer complaint density in the project with reuse: 176/100839.
Rate of error slip-page	QN3 (1997): Percentage of errors that escape unit test.	QN3: The authors found a significantly lower error slip-page rate in the slightly modified reused components: Chi-Square Test p-value for comparison of newly created components and slightly modified reused component is less than 0.0001.
Index of quality of programming	QN4 (2002): It is calculated as relative deviations between development effort and the rework effort required to correct failures detected during the acceptance test: (development effort - the rework effort to correct failures)/development effort.	QN4: Framework reuse is less prone to failures than traditional development. The quality index for development with reuse applications ranges from 0.85 to 0.96. The quality index for development without reuse applications ranges from 0.64 to 0.77.
Average faults per module	QN5 (2005): Module averages (and standard deviations) for total faults, broken down by module origin. The module has four origins: new development, major revision, slight revision, and complete reuse.	QN5: The authors found modules that were completely reused without revision had fewer faults compared to modules that were newly developed, had major revisions, or had slight revisions. The mean value of faults per module: new development 1.28, major revision 1.18, slight revision 0.58, complete reuse 0.02. The standard deviation value of faults per module: new development 2.88, major revision 1.81, slight revision 1.20, complete reuse 0.17.
Number of Delta	QN7 (2001): Where a delta is a change to a software work product such as code. Such a change can be either an enhancement or a repair, and since they have been found to correlate well with faults, deltas are sometimes used for estimating error rates.	QN7: The results showed that Spearman's rank correlation for External Reuse Level (ERL) and Deltas is -0.5, and for External Reuse Frequency (ERF) and Deltas is -1.0. These data indicate a negative relationship between ERF and ERL, and Deltas - the more reuse will produce fewer faults.
Quality rating	QN7 (2001): Developers give a quality rating based on the trouble they had in debugging and maintaining the software. Quality rating is a subjective ordinal measure based on an integer scale ranging from 1-worst to 10-best.	QN7: In total, the authors investigated six modules. Quality ratings varied from 6 to 8, with a median rating of 7. The Spearman's rank correlation indicates a positive correlation (0.46) between External Reuse Level (ERL) and Quality Rating, and a stronger positive correlation (0.62) between External Reuse Frequency (ERF) and Quality Rating. These results showed that higher levels of reuse are related to higher perceptions of quality.
Number of field defects reported by customers	ER17 (2008): The aggregated number of field defects reported by customers.	ER17: Customers reported fewer defects when following systematic reuse - 50% less in systematic reuse compared to ad hoc reuse.
Change in fault rate	ER19 (1995): Reuse effect on the ratio of major errors to total recorded faults at the time of delivery.	ER19: System B and C reused from System A. System B had a 37% improvement in fault rate, and System C had a 33% improvement in fault rate when compared to System A at the time of delivery.

Table 6: Metrics for improved productivity and their definition and results

Metrics	Definition	Results
(Apparent /Gross) Productivity - Size of project per development effort	ER2 (1994): Productivity is calculated as thousand non-comment source statements per engineering month. QN7 (2001): Productivity is calculated as the number of Non-Commentary Source Lines produced per person day (NCSL / Effort). QN10 (2005): Apparent Productivity = (written lines of code (LOC) + reused LOC)/ effort spent in person-hours. Written LOC: "total LOCs that are written when developing the requirement". Reused LOC: "total LOCs that belong to previously developed artifacts and have been used during the development of the new software requirement". QN12 (2012): Gross productivity is defined as the software program size divided by the total direct software development effort. Where software program size is defined as KNCSS (Thousand Non-commentary source statement) and direct software development effort is defined as the person-day of the software developer directly involved in the project). Productivity (PD) = KNCSS / Person-day. QN15 (2014): Productivity rates are measured using the total number of requirements count and total effort in man-hours. Productivity rate = No. of requirements/man-hour. QN15 (2014): Productivity rates are calculated by dividing the total non-comment line of codes by the total effort to develop the product. Productivity rate = SLOC / man-hour ER19 (1995): Productivity is calculated as the average number of lines of code produced per day per person. QN26 (2016): Productivity is measured by the software size of the project to the total effort spent on the project. Software size: Lines of Code (LOC) and total number of Web pages (totPage) are two metrics used for calculating the software size of Web applications. Total effort (Effort): Total effort spent on the project includes the effort from the requirements phase to the maintenance phase. Total effort and maintenance effort are both measured in person days. QN28 (1999): C = Complexity (for New Code and Reused Code), T = Time (for New Code and Reused Code), RAC = Company's average cost of developing software incorporating reuse, relative to the cost of developing without reuse, R = Overall Reuse Rate of the company, PRR = Project Reuse Rate, and DRR = Domain Reuse Rate for the project's domain, $P(\text{productivity}) = [1/(1+RAC) * C_{\text{newcode}}/T_{\text{newcode}} + RAC/(1+RAC) * C_{\text{reusedcode}}/T_{\text{reusedcode}}] * [1-(DRR-PRR)/(1-RAC)/R]$. ER22 (2014): The number of development models per year. QN25 (2015): Gross productivity is defined as the software program size divided by the total direct software development effort. Where software program size is defined as KNCSS (Thousand Non-commentary source statement) and direct software development effort is defined as the person-day of the software developer directly involved in the project). All the time spent on a project, including all training, documentation, source code review, development, and debugging, is counted as the direct software development effort in the productivity calculation. QN5: Total development effort: tenths of human hours.	ER2: One case division got a 57% increase in productivity, and the other case division got a 40% increase in productivity after applying software reuse practices. QN7: The authors investigated two cases. One case showed a marginally positive relationship (0.27) between External Reuse Frequency (ERF) and NSCL/Effort. The other case showed a positive relation (0.31) between External Reuse Level (ERL) and NSCL/Effort. QN10: Comparing three trimesters of the conventional development (CONV) process and Reuse Oriented Development (ROD) process, the author found significantly higher apparent productivity in every trimester of the ROD process and increased apparent productivity in the second and third trimesters of the ROD process. Wilcoxon test p-value of apparent productivity between different trimesters: Trimester 1 (ROD vs. CONV) is 0.0392; Trimester 2 (ROD vs. CONV) is 0.0052; Trimester 3 (ROD vs. CONV) is 0.021. CONV Trimester 1 vs. CONV Trimester 2 is 0.1329; CONV Trimester 2 vs. CONV Trimester 3 is 0.1476; ROD Trimester 1 vs. ROD Trimester 2 is 0.0298; ROD Trimester 1 vs. ROD Trimester 2 is 0.4548. A low p-value between ROD Trimester 1 and ROD Trimester 2 is expected since the repository is not populated, the reuse rate is low, and developers need to learn the reuse process in the first period. QN12: The authors investigated productivity in 30 projects with different verbatim reuse rates. Pearson's correlation coefficient r on productivity against verbatim reuse rate is 0.496, indicating that verbatim reuse has a positive effect on productivity. The p-value of the test is 0.005 also shows a high confidence level of the result. QN15: The authors found after reuse, the productivity rates increased remarkably - product 1:803.2; product 2: 1487.3; product 3: 2796. QN15: The authors investigated four products developed from a product line and found reuse helps improve productivity. The results showed productivity rate increased slightly between products 1 and 2, decreased between products 2 and 3, and increased significantly between 3 and 4. The decrease between products 2 and 3 is because most developers of product 3 were not familiar with the product line development. ER19: System B had a 35% improvement in productivity over system A development. QN26: The projects that adopt reuse approaches had 2% productivity improvement over the previous projects. QN28: The authors investigated three projects with reuse rates of 50.5%, 67.6%, and 76%. The corresponding productivity of the project is 16.5, 20.2, and 38.2. The author concluded that the more reuse, the better the productivity. ER22: After applying software reuse practices, the authors found productivity got 2.5 times greater- before, they implemented 0.8 models per year and now improved to 2.0 models per year. QN25: The authors investigated productivity in 30 projects with different verbatim reuse rates. Pearson's correlation coefficient r on productivity against verbatim reuse rate is 0.476, indicating that verbatim reuse positively affects productivity. The p-value of the test is 0.004 also shows a high confidence level of the result. QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed that newly developed modules (mean, std.dev. and median values are 215.5, 378.0, 110.0) and major revision modules (mean, std.dev. and median values are 193.6, 284.8, 100.0) had the most development effort, while slightly revised modules (mean, std.dev. and median values are 86.2, 144.6, 60.0) had the second most and those completely reused (mean, std.dev. and median values are 5.8, 28.0, 0.0) had the least.
Development Effort		

Table 6 continued from previous page

Metrics	Definition	Results
Development effort per source line/modules/products	QN11 (2005): Effort of developing the game (Man-days)	QN11: The reuse practices help to reduce the development effort - it takes 28 man-days to develop the game with reuse and 88 man-days without reuse.
	ER17 (2008): Total development time (person-month)	ER17: The reuse practices saved 12 person-months of development time (from 32 to 20 person-months).
	ER16 (2011): Initial development time - in hours, for three types of implementation: configurable framework implementation, custom implementations, and migration between the two. Initial development time counts from the start of change implementation and ends when the change is first delivered to the customer. It does not include the time taken to specify or negotiate the change request.	ER16: Custom implementation takes more time (around 195 hours), configurable framework implementation takes less time (around 140 hours), and migration takes the least time (around 120 hours). The average time to create a new scoring model for the customization approach is 24 hours. For the configurable approach, it is 23 hours (including migration time). The average time to implement a new scoring methodology using the configurable framework when the development of the framework is excluded is around four hours. This is a six times increase in efficiency.
	QN5 (2005): The development effort from design specification through acceptance testing in tenths of hours, divided by the number of source lines of code. The development effort includes all design, code, and test efforts, as well as fault correction efforts and change correction efforts. Any post-delivery effort is also included. For modules reused with no, slight, or major revision, the development effort includes only the effort expended on the new project; the effort required to create the original module is not included.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed that newly developed modules had the most effort per source line (mean, std.dev. and median values are 1.089, 3.575, 0.616), major revision (mean, std.dev. and median values are 0.758, 1.976, and 0.421) and slight revision (mean, std.dev. and median values are 0.601, 1.861, and 0.345) modules had the second most and completely reused modules had the least (mean, std.dev. and median values are 0.047, 0.207 and 0.000).
	QN7 (2001): Effort in person days spent per module (effort/module).	QN7: The authors investigated two cases. One case showed a positive relationship (0.66) between External Reuse Frequency (ERF) and effort/module.
Net/Actual productivity	ER18 (2007): Effort spent on each new product (person/day)	ER18: The result showed that the effort spent on each new product reduced by about 35% in 2006 compared to the 2002/03 baseline.
	ER18 (2007): Average time spent on each new product (day or month).	ER18: The result shows that the average time spent on each new product decreased by 26% in 2006 compared to the 2002/03 baseline.
	QN4 (2002): Net productivity = net size of the developed application (object-oriented function points)/hours	QN4: The authors investigated five applications with reuse and four without reuse. The results showed that the applications developed with reuse (2.6, 3.8, 4.53, 5.41, 7.26) provide higher net productivity than the ones developed without reuse (1.06, 1.31, 1.63, 2.56).
	QN10 (2005): Actual Productivity = written LOC/effort spent in person hours.	QN10: Comparing three trimesters of the conventional development (CONV) process and Reuse Oriented Development (ROD) process, the author found a significant difference in actual productivity in the first trimester, resulting from the systematic reuse adoption. After excluding the reuse, the average productivity of a developer in a mature process tends to range within a defined interval. Wilcoxon test p-value of apparent productivity between different trimesters: Trimester 1 (ROD vs. CONV) is 0.0025; Trimester 2 (ROD vs. CONV) is 0.2048; Trimester 3 (ROD vs. CONV) is 0.5696. CONV Trimester 1 vs. CONV Trimester 2 is 0.1329; CONV Trimester 2 vs. CONV Trimester 3 is 0.1476; ROD Trimester 1 vs. ROD Trimester 2 is 0.0792; ROD Trimester 1 vs. ROD Trimester 2 is 0.6639.
	QN15 (2014): Productivity rates are measured using the number of new requirements count and total effort in man-hours. Productivity rate = No. of new requirements/man-hours.	QN15: The authors investigated three products generated from a product line. They found a reduction in productivity between the first and second products (830.2 and 148.7) due to the adaptation period of development with the product line and an increase in productivity between the second and third products (148.7 and 431) as evidence of being trained in development with the product line. The authors also said that future products will have better productivity rates than the first product.
Size of the code	QN5 (2005): The number of source lines of code in the implementation.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed verbatim reuse has the smallest size of code - modules reused with major revision were the largest (mean, std.dev. and median values are 324.1, 221.3, 262.0), newly developed modules were the second largest (mean, std.dev. and median values are 226.7, 158.1, 185.0), those reused with slight revision were the third largest (mean, std.dev. and median values are 192.8, 125.7, 164.5), and those completely reused without revision were the smallest (mean, std.dev. and median values are 146.2, 152.3, 92.0).
Estimated development effort	ER22 (2014) Actual/Estimated effort: total man-hours (month or year) spent in developing the core assets and N models	ER22: By using the reuse practice, three instrument models took 18 months (1.5 years) to develop, while the estimated time for two instrument models is 2.5 years.
Percentage of development effort spent on design	QN5 (2005): The effort spent during module design activities divided by total module development effort. Any modules reused without revision or with zero module development effort are excluded from this metric calculation.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. They found modules that were newly developed (mean, std.dev. and median values are 19.8, 24.7, 9.5) had a higher percentage of effort spent in module design than did all other modules - major revision modules: mean, std.dev. and median values are 15.7, 24.6, 0.0; slight revision modules: mean, std.dev. and median values are 12.3, 22.2, 0.0; complete reuse modules: mean, std.dev. and median values are 14.6, 30.3, 0.0.
Testing effort	ER17 (2008): Total testing time (person-month)	ER17: Using reuse practices saved three person-months for quality assurance (mainly testing, from eight to five person-months).

Table 7: Metrics for better maintainability and their definition and results

Metrics	Definition	Results
Fault correction effort	QN5 (2005): The fault correction development effort is measured per source line in tenths of hours. ER16 (2011): Hours spent on bug fixing by year.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The authors found the modules reused without revision had the lowest fault correction effort. The modules reused with major revision had the highest fault correction effort. ER16: The results showed that overall the required maintenance in 2010 was eight times less than the previous year for the configurable reuse approach.
Code modification rate(MOD)	QN9 (2004): Size of new or modified code divided by the total size of the component. QN13 (2008): Size of the modified code between releases divided by the total size of the code.	QN9: The authors compared the MOD between the reused and non-reused components. The results showed that blocks are 49% modified totally, 43% for reused, and 57% for non-reused ones (with KLOC). The gradient of the regression line is larger for non-reused blocks, indicating that they are modified more than reused ones. The authors also performed t-tests (a two-tail t-test p-value= 0.001, and a one-tail t-test p-value=0.999) and concluded that non-reused components are more modified than reused ones. QN13: The authors investigated MOD in reused and non-reused components between Release 2 and 3. The results showed that reused components are less modified than the non-reused ones, and this trend was confirmed across releases. The mean, median, and standard deviation values for MOD in reused components are 0.43, 0.43, and 0.14. The mean, median, and standard deviation values for MOD in non-reused components are 0.57, 0.60, and 0.14.
Rework effort	QN3 (1997): Dividing rework effort by the number of source statements.	QN3: The authors found reuse via slight modification shows a 35% reduction in rework cost over newly created components, while verbatim reuse provides an 88% reduction.
Fault isolation effort	QN5 (2005): The fault isolation development effort is measured per source line in tenths of hours.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed that the modules reused verbatim had the lowest fault isolation effort. Mean and std.dev. values for major revision modules are 23.10 and 44.50. Mean and std.dev. values for new development modules are 21.10 and 52.10. Mean and std.dev. values for slight revision modules are 10.60 and 26.70. Mean and std.dev. values for complete reuse modules are 0.20 and 3.40.
Change correction effort	QN5 (2005): The change correction development effort is measured per source line in tenths of hours. Changes are sometimes made to "modules reused without revision" because of improved documentation or other changes that do not modify functionality.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed that modules reused verbatim had the lowest change correction effort.
Changes per source line	QN5 (2005): The number of changes divided by the number of source lines of code.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision, and complete reuse. The results showed that modules reused verbatim had the least changes per source line.
Average number of changes	QN5 (2005): The number of changes per module.	QN5: The authors have divided all modules into four categories: new development, major revision, slight revision and complete reuse. The authors found the modules reused verbatim had the least changes per module (mean and std.dev. values are 0.08 and 0.40). The mean and std.dev. values of new development modules are 1.37 and 2.29. The mean and std.dev. values of slight revision modules are 1.02 and 1.34. The mean and std.dev. values of major revision modules are 2.40 and 2.33.
Number of codes reduced	QN11 (2005): The reduced code's size compared to the code's original size.	QN11: 26% of the code was reduced compared to the original code.

Table 8: Metrics for cost saving and their definition and results

Metrics	Definitions	Results
Net-present-value	ER2 (1994): Net present value takes the estimated value of reuse benefits and subtracts from its associated costs, taking into account the time value of money. It includes metric time horizon, start/up resources required, ongoing resources required, gross cost, gross savings, return on investment(saving/cost), net present value, break-even year(recoup start-up costs)	ER2: After reuse, one division got 410% savings, and the other division got 216% savings.
The development effort in man-days	ER6 (2005): Calculating the development effort in man-days for different types of development methods: Systematic Reuse (SR), New Development (ND), Opportunistic Reuse (OR), and Controlled Reuse (CR).	ER6: Among all the development methods, the projects and modules had the least development effort in SR (asset J: 1660 man-days, asset L: 133 man-days) and the most in ND (asset J: 3600 man-days, asset L: 240 man-days). Other than SR and ND, CR (asset J: 2472 man-days, asset L: 195 man-days) took lesser development effort than OR (asset J: 3120 man-days, asset L: 234 man-days).
Savings of systematic reuse over new development	ER6 (2005): Calculate the percentage of savings between systematic reuse over new development.	ER6: Project J saved 54%, and Project L saved 45% from systematic reuse over new development.
Savings in development days/man-months/man-days	ER6 (2005): Calculate the development effort difference using development days/man-months/man-days between reuse and non-reuse approaches.	ER6: Project J saved 2000 development days and 100 man-months. Project L saved 97 development days and five man-months.

Table 8 continued from previous page

Metrics	Definitions	Results
Direct effort (in person-hours)	ER8 (2004): Systematic Reuse cost (adapted as-sets)= Cmc+Cnr+n*Cbb; Controlled Reuse cost =n*(Ccp+Cwb); Pure Development cost =n*Cnd; mc=mining and cataloging; nr=new for reuse; bb=black-box reuse; cp=copy and paste; wb=white-box reuse; nd=new development; n=number of reuses; Saving percentage between the cost of reuse scenario S and the cost of alternative reuse scenario S': (CS' - CS)/CS'.	ER8: For all seven assets, systematic reuse or controlled reuse saves a lot from new development. For five assets, systematic reuse gave savings between 42% to 81% compared to new development. Controlled reuse saved the most in one asset, while systematic reuse saved the most in the rest six assets.
Costs	QN12(2012): Total Cost = Cost of management and support shared based on the duration of the project + Cost of direct effort for reuse and development of the software. Cost of direct effort for reuse and development of the software = Effort in person-day $\times$ Salary rate of developer. QN25(2015): The total costs include direct costs and indirect costs. The direct costs are the salaries of the developers. The indirect costs are the salary of the management and support team. Total Cost Index = Direct Cost Index + Indirect Cost Index. Direct Cost Index = Normalised Cost of direct effort for reuse and development of the software = (Salary Rate of Developer $\times$ Effort in person - day) / KNCSS. Indirect Cost Index = 0.306 $\times$ Normalised Duration = (0.306 $\times$ Duration in Calendar Days) / KNCSS. The Salary Rate of Developer is calculated by acquiring the direct salary of different developers that reflects their experience in the field. The second part is an indirect cost that accounts for the shared management and support cost. The management and support team salary and expenditure are shared with all projects handled by the team according to project duration (PD).	QN12: The authors investigated the total cost index for 30 projects with different verbatim reuse rates. The Pearson's Pearson Coefficient r on verbatim reuse rate against total cost index is -0.471, indicating that cost will reduce with a higher verbatim reuse rate. The p-value of the test is 0.009 also shows a high confidence level of the result. QN25: The authors investigated the total cost index for 30 projects with different verbatim reuse rates. Pearson's correlation coefficient r on the verbatim reuse rate against the total cost index is -0.505, indicating that cost will reduce with a higher verbatim reuse rate. The p-value of the test is 0.002 also shows a high confidence level of the result.
The breakeven point	ER16 (2011): The point that the reuse benefits have paid the costs of implementing reusable assets.	ER16: It was achieved in the second year of the evolution after only six implementations.
Cumulative development costs	ER24 (2011): The comparison between the cumulative development costs for products with and without Product Line Engineering(PL). The cumulative cost without PL was estimated based on the previous projects.	ER24: The result shows PL paid off the initial investment for this project by restricting core asset creation to a limited number ⁷ of features.
Return on investment	ER27 (1998): For this application, a 1% improvement in the level of reuse achieved meant a saving in excess of £100,000.	ER27: The reuse from project A to B achieved a level of 82% code reuse with a corresponding 80% cost saving. Reuse was, therefore, capable of providing a convincing return on investment.

Table 9: Metrics for faster time-to-market and their definition and results

Metrics	Definitions	Results
Actual/ Estimated calendar month	ER2 (1994): Time in calendar months.	ER2: The actual time (with reuse) - 21 calendar months took less than the estimated one (without reuse) - 36 calendar months.
Delivery time	ER19 (1995): Time takes to deliver the system - day or month.	ER19: Versions of System C (with reuse) had a shorter delivery time than versions of System A. Delivery delays for System C versions range from 0 to 13 days, while they range from 56 to 107 days for System A versions. System C delivered 30% faster delivery time compared to other systems at the time and had 44% improvement compared to version 1 of system A.
Development time	QNL20 (2015): Development time for any artifact is calculated by using the start and end timestamps for any work contributing to the development. The savings from the reuse of any building block are estimated as the building block's known past development time minus the amount of time spent determining whether or not to do the reuse.	QNL20: Reuse of requirements from product A reduced the time to market each of its reusing products to 60 % of the estimated time to market.
Development period	ER24 (2011): The time used to develop products.	ER24: The development period required for the product with software reuse was shortened by 60% compared to that of the first product.

Table 10: Metrics for other benefits and their definition and results

Others Benefits	Metrics	Definition	Results
Increased performance	Running time and memory usage	QN11 (2005): The authors measured the running time with a memory monitor and profiler turned on in the Wireless Toolkit.	QN11: The memory usage of the game developed with reuse decreased by almost 19%, and the game ran almost 10% faster than the original games without reuse. Both are the best cases.
	Resource consumption	ER17 (2008): resource consumption of an embedded system: the usage of ROM (i.e., the size of the binary program code).	ER17: The XENON8 (platform with reuse) had less usage percentage than XENON7 (platform without reuse) - XENON8: 48%, 52%, 74%, and XENON7: 96%-97%.
Improved learning	Proxy of programmer's learning	QN4 (2002): It was calculated as the cumulated size of developed software (Object Oriented Function Points) up to the i-th application.	QN4: The learning rate in development with reuse is more than double (2.46) that learning in development without reuse.

⁷It was hard to extract the exact number from the figure.

Table 10 continued from previous page

Others Benefits	Metrics	Definition	Results
Prolonged lifecycle of reused product (game)	Grid Analysis (daily activity levels in the Game per 1 square km grid)	QN29 (2021): Average Actions per grid and day, Pre-new game release, Average Actions per grid and day, Post-new game release, Average number of old game GIP per grid, Average number of new game GIP per grid. The grid analysis considers only 7675 of our total 25,020 grids, as only 7675 grids have interaction points in the old game. Accordingly, in-game activity for the old game can only be observed in these 7675 grids	QN29: The new game's introduction relates to an increase in the number of active players in the old game by 32%, while the number of new players joining the old game multiplied 16-fold.

Table 11: Metrics for all costs and their definition and results

Costs	Metrics	Definition	Results
Reduced quality	Error source & error type	QN3 (1997): Error sources: requirement, functional specification, design, code, or previous change. Type of errors: procedural error - a computational or a logic error; interface error - an internal or external interface error; data error - an initialization or a data value error.	QN3: The study found increased design errors in modified reusable components, increased requirements and specification errors in new components, and increased errors due to a previous change in verbatim reused components. Slightly modified reusable components have more interface errors.
	Defect density per defect type	QN21 (2009): Divided Non-commented Source Lines of Code (NSLOC) of each system by the number of defects to calculate the defect density. The NSLOC was counted using the Eclipse tool because that is the development tool used in the company. The types of defects are Function, Assignment, Interface, Algorithm, Data, GUI, Relationship, Checking, Documentation, Timing/Serialization.	QN21: For the reused framework (JEF) and two applications (DCF and S&A) which are reusing framework, Interface and GUI defects have the highest density. Comparing JEF and DCF, DCF has the highest density in function, relationship, checking, and data defects. Comparing JEF and S&A, S&A has the highest density in function, data, checking, and algorithm defects.
	Defect density per defect severity	QN21 (2009): Divided the number of defects of different severity by the Non-commented Source Lines of Code (NSLOC). The severity of defects is: Critical, High, Medium, Low, and Very low.	QN21: The most severe defect types in the reused framework (JEF) are interface and assignment. The most severe defect types in the application (DCF) reusing JEF are relationship and function. The most severe defect types in the application (S&A) reusing JEF are algorithm and function.
	Defect density per impact	QN21 (2009): Divided the number of different impacts of defects by Non-commented Source Lines of Code (NSLOC). Defects based on impact attributes of Orthogonal Defect Classification (ODC) are installability, integrity/security, performance, maintenance, serviceability, migration, documentation, usability, standards, reliability, and capability.	QN21: The most common impacts of defects are capability and usability.
	Complexity	QN11 (2005): Complexity of the design: MSOO - Maximum Size of Operation; MNOL - Maximum Number Of Levels; MNOP - Maximum Number Of Parameters; NOM - Number Of Members; NORM - Number Of Remote Methods. Polymorphism: NOOM - Number Of Overridden Methods.	QN11: The complexity was increased a bit as the MSOO (hero component went ten from two) and MSOR value increased after using a reuse technique.
Reduced maintainability	Efforts to isolate and complete each error correction	QN3 (1997): The authors categorized the effort to isolate and complete each error correction into three categories: less than one hour, one hour to one day, one to three days, and more than three days.	QN3: The results showed that reused verbatim components had the highest percentage of errors requiring more than one day to complete an error correction, and the new components had the lowest percentage, while the modified components fell in between. Chi-Square Test p-value for difficult isolation (more than one day or isolation effort) comparison of new development vs. extensively modified is 0.4728, new development vs. slightly modified is 0.1756, new development vs. verbatim is 0.3166, extensively modified vs. slightly modified is 0.1302, extensively modified vs. verbatim is 0.4966, slightly modified vs. verbatim is 0.1396. Chi-Square Test p-value for difficult completion (more than one day of completion effort) comparison of new development vs. extensively modified is 0.3620, new development vs. slightly modified is 0.1243, new development vs. verbatim is 0.1063, extensively modified vs. slightly modified is 0.0508, extensively modified vs. verbatim is 0.2629, slightly modified vs. verbatim is 0.0195.
	Metrics about complexity and structuredness	QN23 (2006): Not reported	QN23: The metrics of complexity and structuredness results indicate that reuse has a negative impact on maintainability and reusability.
Performance decay	Number of cycles (NoCycles)	QN15 (2014): The number of cycles (NoCycles) for the modules to receive the command from the system and send the corresponding data.	QN15: The authors investigated NoCycles for three modules with different reuse rates. The results showed that the number of cycles increases with increasing reuse, which resulted from the reuse of the middleware. The reuse rate and NoCycles of Module 1 are 0% and 7914. The reuse rate and NoCycles of Module 2 are 81% and 9756. The reuse rate and NoCycles of Module 1 are 52% and 9648.

Table 11 continued from previous page

Costs	Metrics	Definition	Results
	Time delay(ms)	QN15 (2014): Separately for three scenarios (before SPL, SPL with pull strategy, SPL with push strategy), the delay from reception of the track data from the system.	QN15: The results showed that the performance time was delayed further when the interface was changed to a more reusable and abstract level. The system developed before SPL had the least time delay (minimum delay 0.85ms, maximum delay 1.05ms, and average delay 0.9ms). The system developed with the push strategy had less time delay (minimum delay 2.12ms, maximum delay 5.5ms, and average delay 3.2ms) compared to the pull strategy (minimum delay 2.12ms, maximum delay 35ms, and average delay 20ms). QN15: The results showed that the CPU required more usage when the interface was changed to a more reusable and abstract level. The system developed before SPL had the lowest percentage of CPU usage (72.3%). The system developed with the push strategy (79.9%) had less percentage of CPU usage compared to the pull strategy (81.5%).
	CPU usage (%)	QN15 (2014): The CPU usage during the scenario (before SPL, pull strategy, push strategy).	
Reduced productivity	Size of code per development effort	QN7 (2001): Productivity is calculated as the number of Non-Commentary Source Lines produced per person day (NSCL / Effort).	QN7: The authors investigated two cases. One case showed a negative relationship (-0.22) between External Reuse Frequency (ERF) and NSCL/Effort. The other case showed a marginally negative relation (-0.17) between External Reuse Level (ERL) and NSCL/Effort. QN7: The authors investigated two cases. One case showed a negative relationship (-0.45) between External Reuse Level (ERL) and effort/module. The other case showed both negative relationships (-0.69) between External Reuse Frequency (ERF) and effort/module and (-0.22) between External Reuse Level (ERL) and effort/module.
	Development effort per module	QN7 (2001): Effort in person days spent per module (effort/module).	
Low conformance in architecture	conformance	QN23(2006): Conformance is expressed in terms of percentage and calculated by the number of convergences divided by the total number of dependencies (i.e., divergences plus convergences).	QN23: The proposed product line architecture is not fully conformant, and the product with the product line approach has the lowest conformance. The authors investigated the conformance between the product architecture and the product line architecture. The results showed that developing reusable components of a concrete project poses the risk of violating the initially defined product line architecture, and the concrete products' architecture did not fully conform to the product line architecture - the products' average conformance to the layering is 86.01% and the products' average conformance to the framework is 78.13%.

Appendix G Quality assessment results

Table 13: Quality assessment results

Paper ID	Context	Research design	Sampling	Control group	Data collection	Data analysis	Reflexivity	Findings	Value of the research	Total score
QN1	1	0.5	0.75	1	0.75	0.75	1	0.75	0.75	7.25
QN3	1	0	0.5	1	1	0.75	0	0.75	0.75	5.75
QN4	0.75	0.5	0.5	1	0.5	0.75	1	0.75	0.75	6.5
QN5	0.75	0	0.5	1	1	0.75	1	0.75	1	6.75
QN7	1	0.5	0.75	0.5	0.75	0.75	0	0.5	0.75	5.5
QN9	1	0.5	1	1	0.75	0.75	0	1	1	7
QN10	1	1	0.75	1	0.5	0.75	1	0.75	0.75	7.5
QN11	1	0.5	0.5	1	0.25	0.25	0	0.5	0.25	4.25
QN12	1	0.5	0.75	1	0.5	0.75	0	0.5	1	6
QN13	1	1	0.75	1	1	0.75	0.75	1	1	8.25
QN15	1	0	0.75	0.5	0.5	0.5	0	0.75	0.25	4.25
QN21	1	0.5	0.75	1	0.5	1	1	1	1	7.75
QN23	1	0	0.25	1	0.25	0.5	0	0.5	0.75	4.25
QN25	1	0.5	0.75	1	0.25	0.5	0	0.5	1	5.5
QN26	0.5	0.5	0.25	1	0.25	0.5	0.5	0.75	0.5	4.75
QN28	1	0	0.75	0.5	1	0.75	0.5	0.75	0.75	6
QN29	0.75	0	0.75	1	0.5	0.75	0	1	1	5.75
ER2	1	-	0.5	1	0.25	0.75	-	0.5	0	4
ER6	1	-	0.5	1	0.25	0.75	-	0.25	0.5	4.25
ER8	0.75	-	0.5	1	0.25	0.5	-	0.25	0.75	4
ER16	1	-	1	1	0.25	0.75	-	0.25	0.75	5
ER17	1	-	0.75	1	0.25	0.5	-	0.75	0.75	5
ER18	1	-	0.5	1	0.25	0.5	-	0.5	0.5	4.25
ER19	1	-	1	1	0.25	0.5	-	0.5	0.25	4.5
ER22	0.75	-	0	1	0.25	0.25	-	0.25	0.75	3.25
ER24	1	-	0.75	1	0.25	0.5	-	0.25	0.75	4.5
ER27	1	-	0.25	1	0.25	0.25	-	0.5	0.25	3.5
QL14	1	0.5	1	-	0.25	0.5	0	0.75	1	5
QL30	0.25	0.5	0.25	-	0.25	0.25	0	1	0.25	2.75
QNL20	1	1	0.75	1	0.75	0.5	0	1	0.25	6.25